

This homework has 10 questions, for a total of 100 points and 3 extra-credit points.

Unless otherwise stated, each question requires *clear*, *logically correct*, and *sufficient* justification to convince the reader.

We strongly encourage you to typeset your solutions in L^AT_EX.

If you collaborated with someone, you must state their name(s). You must *write your own solution* for all problems and *may not use any other student's write-up or any AI tools*.

0. **Before you start; before you submit.**

If applicable, state the name(s) and username(s) of your collaborator(s).

Solution: No collaborators.

- (10 pts) 1. **Self assessment.** Carefully read and understand the posted solutions to the previous homework. Identify one part for which your own solution has the most room for improvement (e.g., has unsound reasoning, doesn't show what was required, could be significantly clearer or better organized, etc.). Copy or screenshot this solution, then in a few sentences, explain what was deficient and how it could be fixed.

(Alternatively, if you think one of your solutions is significantly *better* than the posted one, copy it here and explain why you think it is better.)

If you didn't do last week's homework, choose a problem from it that looks challenging to you, and in a few sentences, explain the key ideas behind its solution in your own words.

Solution: I chose the minimum-subarray-product problem from the previous homework as a useful example to reflect on. Every nonempty subarray ending at position i is either the one-element subarray $[A_i]$, or is obtained by appending A_i to a subarray ending at $i - 1$. Thus, if E_i denotes the smallest product of a nonempty subarray ending at i , then

$$E_i = \min\{A_i, E_{i-1}A_i\}, \quad E_1 = A_1.$$

The global answer is $\min_i E_i$. My earlier write-up did not clearly separate this local state from the global minimum and did not explain why the two cases exhaust all subarrays ending at i . A better solution states the base case, proves the two-case decomposition explicitly, and then gives the $O(n)$ running time from the single pass through the array.

2. **Keeping things fresh with potpourri.**

- (3 pts) (a) Consider the following algorithm STOOGESORT. Give, with brief justification, a recurrence for the running time $T(n)$ as a function of the input array size n . Then, use the Master Theorem to derive a closed-form solution for the running time. You may ignore off-by-one errors introduced by the ceiling $\lceil 2n/3 \rceil$ since it won't affect the asymptotic running time.
Ungraded Note: It turns out STOOGESORT is a correct sorting algorithm! Maybe you can figure out why.

```

1: function STOOGESORT( $A[1, \dots, n]$ )
2:   if  $n = 1$  then
3:     return  $A$ 
4:   if  $n = 2$  and  $A[1] > A[n]$  then
5:     swap  $A[1]$  and  $A[n]$ 
6:   if  $n > 2$  then
7:      $t \leftarrow \lceil 2n/3 \rceil$ 
8:     STOOGESORT( $A[1, \dots, t]$ )
9:     STOOGESORT( $A[n - t + 1, \dots, n]$ )
10:    STOOGESORT( $A[1, \dots, t]$ )
11:  return  $A$ 

```

Solution: For $n > 2$, the algorithm makes three recursive calls, each on an array of size $\lceil 2n/3 \rceil$, and all other work is constant. Therefore, ignoring the harmless ceiling as permitted,

$$T(n) = 3T(2n/3) + \Theta(1), \quad T(1), T(2) = \Theta(1).$$

In the Master Theorem notation, $a = 3$, $b = 3/2$, and $f(n) = \Theta(1)$. Since

$$n^{\log_b a} = n^{\log_{3/2} 3} \quad \text{and} \quad \Theta(1) = O\left(n^{\log_{3/2} 3 - \varepsilon}\right)$$

for any $0 < \varepsilon < \log_{3/2} 3$, this is case 1 of the theorem. Hence

$$T(n) = \Theta\left(n^{\log_{3/2} 3}\right) = \Theta\left(n^{\ln 3 / \ln(3/2)}\right).$$

The exponent is approximately 2.7095.

- (3 pts) (b) Consider the following function, which takes as input non-negative integers a and b that are powers of two.

```

1: function ALG( $a, b$ )
2:   if  $a = 1$  or  $b = 1$  or  $a = b$  then return 0
3:   if  $a > b$  then return ALG( $a/2, 2b$ )
4:   if  $b > a$  then return ALG( $2a, b/2$ )

```

Either find a valid potential function to prove that ALG halts on all valid inputs a, b , or show that no such function exists by giving an input on which ALG runs forever with brief justification.

Solution: No such termination proof can hold for all valid inputs, because the algorithm can enter a two-cycle. Take $(a, b) = (2, 4)$. The first call has $b > a$, so it makes the recursive call

$$\text{ALG}(2, 4) \mapsto \text{ALG}(4, 2).$$

In the second call $a > b$, so

$$\text{ALG}(4, 2) \mapsto \text{ALG}(2, 4).$$

Neither pair satisfies $a = 1$, $b = 1$, or $a = b$, so the base case is never reached. Thus the computation runs forever on this valid input, and a potential function proving termination on every valid input cannot exist.

- (3 pts) (c) State, with brief justification, if this algorithm is “efficient” with respect to the input size. You may assume that the divisibility check takes polynomial time with respect to the input size (e.g., using the grade-school division algorithm).

```

1: function ISPRIME( $k$ ) //  $k \geq 2$  is a positive integer
2:   for  $i = 2, \dots, \lfloor \sqrt{k} \rfloor$  do
3:     if  $k$  is divisible by  $i$  then return false
4:   return true

```

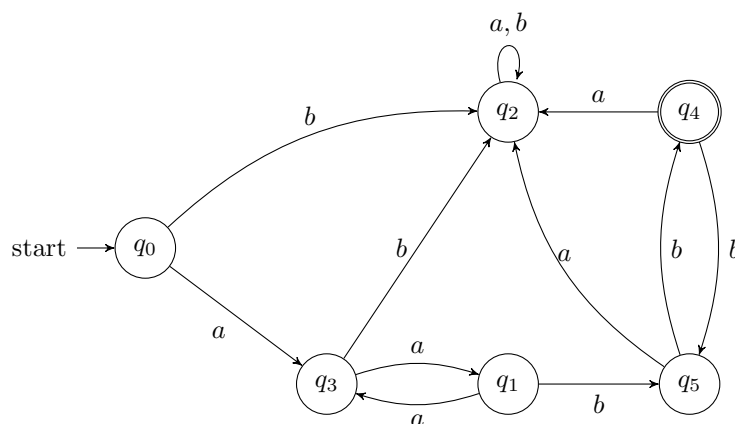
Solution: The standard input encoding for an integer is binary, so if the input has length m , then $k < 2^m$ and $\log k = \Theta(m)$ (up to the usual constant conventions for small k). On a prime input the loop performs $\lfloor \sqrt{k} \rfloor - 1 = \Theta(\sqrt{k})$ divisibility tests. Thus the worst-case running time is

$$\Theta(\sqrt{k}) \text{ poly}(\log k) = 2^{\Theta(m)} \text{ poly}(m),$$

which is exponential rather than polynomial in the input length. The algorithm is therefore not efficient with respect to the usual binary encoding, even though each individual divisibility test is polynomial-time.

3. DFA warmup.

Consider the DFA in the figure below.



DFA for Question 3

- (5 pts) (a) Recall that a DFA is formally defined as a 5-tuple $(\Sigma, Q, q_0, F, \delta)$. Give these five components for this DFA (Question 3), with the state-transition function presented in table format, as in lecture.

Solution: The alphabet is $\Sigma = \{a, b\}$, the state set is

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\},$$

the start state is q_0 , and the accepting set is $F = \{q_4\}$. The transition function is

δ	a	b
q_0	q_3	q_2
q_1	q_3	q_5
q_2	q_2	q_2
q_3	q_1	q_2
q_4	q_2	q_5
q_5	q_2	q_4

Hence the complete DFA is $(\Sigma, Q, q_0, F, \delta)$ with the components listed above.

- (3 pts) (b) Run the DFA on the following input strings, and list all the ones it accepts. (No justification is needed.)

- ε
- ba
- $aabb$
- $aabbbb$
- $aababba$
- $aaaabb$
- $aaabbbb$

Solution: The accepted strings in the list are

$$aabb, \quad aabbbb, \quad aaaabb.$$

- (4 pts) (c) Give, with brief justification, the language that this DFA decides, expressed in ‘string notation’ (alphabet characters; ‘power’, star, and plus superscripts; and the $|$ operator).

Solution: From q_0 , an input must begin with a ; the only way to avoid the sink state while reading the initial block of a ’s is to read an even, positive number of a ’s, ending in q_1 . The first b then moves to q_5 , and the b -transitions alternate between q_5 and the accepting state q_4 . Any a after a b goes to the sink state. Therefore

$$L = (aa)^+(bb)^+.$$

Equivalently, $L = \{a^{2r}b^{2s} : r, s \geq 1\}$.

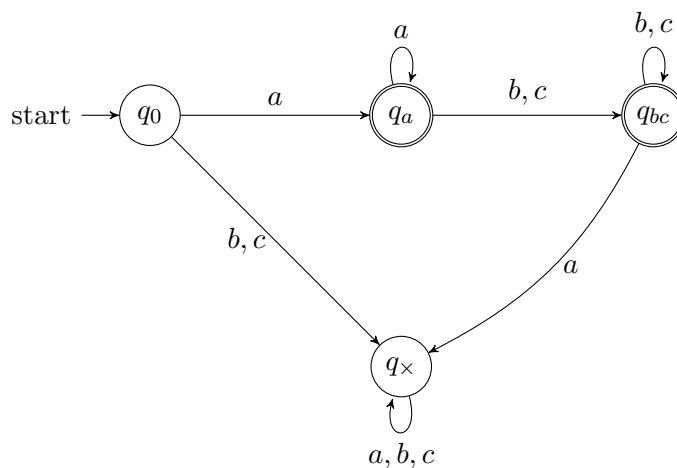
4. Constructing DFAs.

For each of the following languages, draw a DFA that decides it. Briefly justify your answers. Use no more than 8 states.

If you're writing your solution in $\text{\LaTeX}$, you might want to check out [this tool](#) to generate `tikzpicture` script for your DFA.

- (4 pts) (a) The language over alphabet $\{a, b, c\}$ corresponding to $a^+(b|c)^*$.

Solution: The following four-state DFA records whether the required initial block of a 's has begun and whether a b or c has already appeared.

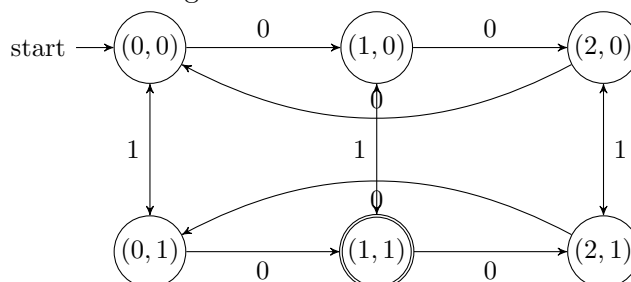


Here q_0 is the start state, q_a and q_{bc} are accepting, and $q_\times$ is a rejecting sink. Thus the machine accepts exactly a nonempty initial block of a 's followed by any number of b 's and c 's.

- (7 pts) (b) The language of binary strings in which the number of 0s mod 3 is 1 *and* the number of 1s mod 2 is 1.

For example, the strings 01, 01000, and 1110 are in this language, while ϵ , 1, 001, 0011, and 000111 are not.

Solution: Use one state for each pair (i, j) , where i is the number of 0's modulo 3 and j is the number of 1's modulo 2. The start state is $(0, 0)$ and the only accepting state is $(1, 1)$. On input 0 the first coordinate increases modulo 3; on input 1 the second coordinate toggles. The resulting six-state DFA is:



In every state, the label records exactly the two relevant residues, so the accepting

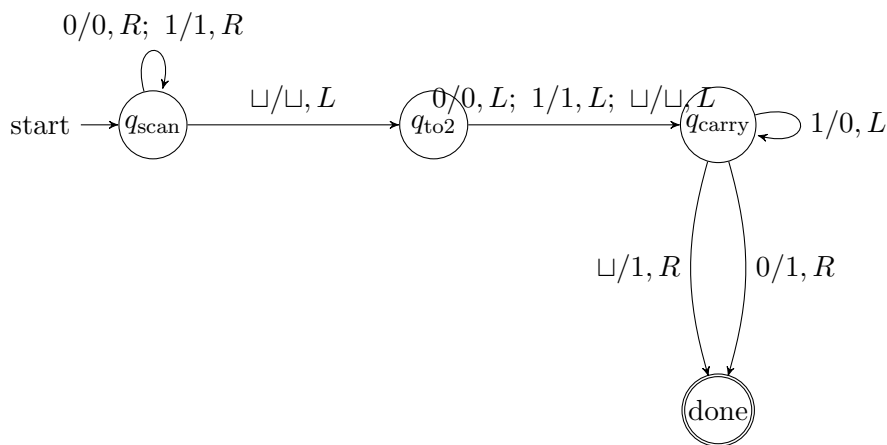
condition is precisely the one stated in the problem.

5. Turing machines.

- (7 pts) (a) Go to <https://turingmachine.io/> and step through the example code titled “binary increment,” which adds 1 to an input integer (represented as a binary string). Modify this machine to add 2 instead of 1 (scroll down on the page for instructions on how to write the code). Give a picture or screenshot of your new machine as your solution. You do not need to provide justification.

Note: The TM model from lecture and TM model used by turingmachine.io are slightly different. The TM model used in class has a one-way infinite tape and accept/reject states. On the other hand, the TM model using by turingmachine.io has a two-way infinite tape and no explicit accept/reject states; instead, the output is on the tape when the ‘done’ state is reached.

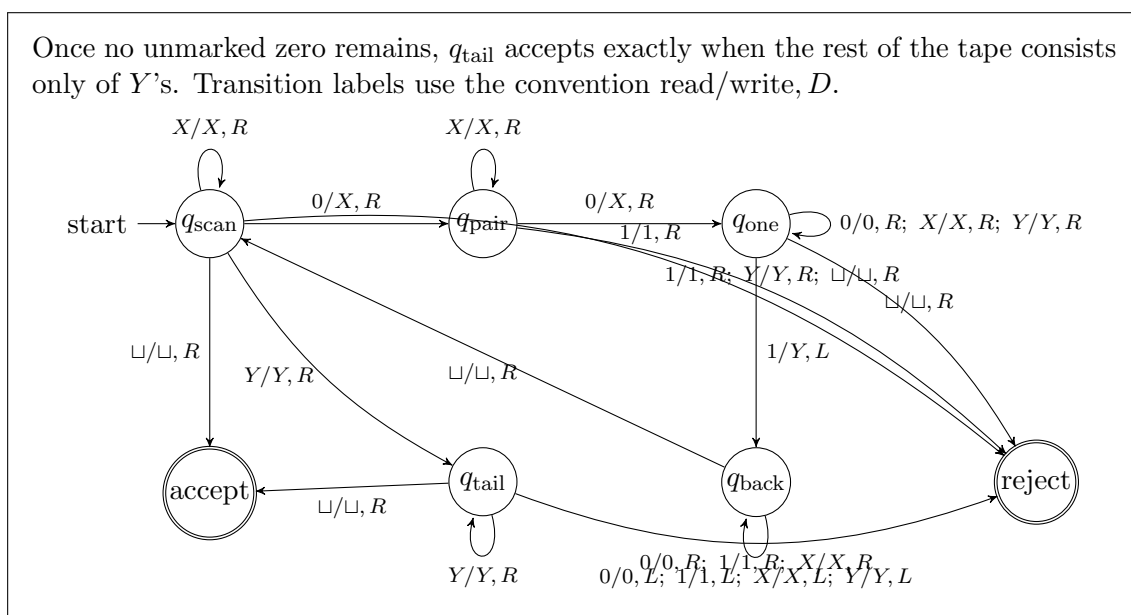
Solution: The diagram below is a direct “add two” machine. A label $x/y, D$ means “read x , write y , move in direction D ”; $\sqcup$ is the blank symbol. The head starts on the most significant input bit. The machine scans to the right end, moves to the second least significant bit, and performs one binary increment there. If a carry runs past the most significant bit, it writes a new leading 1.



- (10 pts) (b) Let L be the language $\{0^{2k}1^k : k \geq 0\}$ over the alphabet $\{0, 1\}$.

Draw a state-machine diagram for a Turing machine that decides L . This drawing can be done however you like (using <https://turingmachine.io/>, drawn in latex, hand-drawn, etc.), as long as it is readable. No justification is necessary. Your machine should follow the lecture convention of having explicit accept and reject states and halt in one of them on every input. You may use turingmachine.io to construct the diagram, but you should label the two halting states accept and reject.

Solution: The machine uses X to mark a used 0 and Y to mark a used 1. Each pass marks two zeros and then one one. The state q_{scan} finds the leftmost unmarked zero; q_{pair} finds its partner; q_{one} finds an unmarked one; and q_{back} returns to the left end.



6. Decidability.

Recall that to prove that a problem is decidable, it suffices to provide pseudocode, and prove its correctness. It is not necessary to explicitly describe a Turing Machine that solves the problem, since any problem solvable by pseudocode can be solved by a Turing Machine, and vice versa.

- (5 pts) (a) Prove any finite language $L = \{\ell_1, \dots, \ell_n\}$ is decidable.

Solution: Define the following decider, where equality means equality of the entire input string.

```

1: for  $i \leftarrow 1$  to  $n$  do
2:   if  $w = \ell_i$  then
3:     return accept
4: return reject

```

Algorithm 1: FiniteLanguageDecider(w)

The algorithm halts because the list has finitely many entries and each string comparison examines only finitely many symbols. If $w \in L$, then $w = \ell_i$ for some i , so the algorithm accepts. Conversely, if it accepts, it found an index i with $w = \ell_i$, which implies $w \in L$. Thus the algorithm decides L .

- (6 pts) (b) Let $L = (L_1 \setminus L_2) \cup L_3$, where L_1, L_2 , and L_3 are *decidable* languages. Prove that L is decidable.

Note: $A \setminus B := \{x \in A : x \notin B\}$ denotes the *set difference* between A and B .

Solution: Let D_1, D_2, D_3 be deciders for L_1, L_2, L_3 , respectively. On input x , run all three deciders and record their Boolean answers:

```

1:  $b_1 \leftarrow D_1(x)$ 
2:  $b_2 \leftarrow D_2(x)$ 
3:  $b_3 \leftarrow D_3(x)$ 
4: if ( $b_1 = \text{accept}$  and  $b_2 = \text{reject}$ ) or  $b_3 = \text{accept}$  then
5:   return accept
6: else
7:   return reject

```

Algorithm 2: $\text{Decide}(L_1 \setminus L_2) \cup L_3$

Each D_i halts, so this algorithm halts. Moreover, b_1 is accept exactly when $x \in L_1$, b_2 is reject exactly when $x \notin L_2$, and b_3 is accept exactly when $x \in L_3$. Therefore the condition in the final test is equivalent to $x \in (L_1 \setminus L_2) \cup L_3$, proving correctness.

7. Pac-Man's journey.

Pac-Man lives on a two-dimensional grid that extends infinitely in all directions, and its moves are controlled by a mysterious outside force. Movement commands come from the alphabet $\Sigma = \{L, R, U, D\}$, which correspond to moving 1 unit left, right, up, and down, respectively. Pac-Man starts from home at the origin $(0, 0)$, and receives an arbitrary (finite) string of commands $s \in \Sigma^*$ to execute in sequence.

- (6 pts) (a) Define an *even position* in the grid as one in which both the horizontal and vertical coordinates are even numbers. Give, with brief justification, a DFA that determines whether Pac-Man would end up at an even position after completing the sequence of commands in s . If so, the DFA should accept s ; otherwise, it should reject s .

Solution: Let q_{ij} represent the parity state in which the horizontal coordinate is $i \pmod{2}$ and the vertical coordinate is $j \pmod{2}$. The DFA is

$$Q = \{q_{00}, q_{10}, q_{01}, q_{11}\}, \quad q_0 = q_{00}, \quad F = \{q_{00}\},$$

with transition table

δ	L	R	U	D
q_{00}	q_{10}	q_{10}	q_{01}	q_{01}
q_{10}	q_{00}	q_{00}	q_{11}	q_{11}
q_{01}	q_{11}	q_{11}	q_{00}	q_{00}
q_{11}	q_{01}	q_{01}	q_{10}	q_{10}

A horizontal move toggles the first parity bit and a vertical move toggles the second. Starting from $(0, 0)$, the state after reading s therefore records exactly the parities of Pac-Man's final coordinates, so the sole accepting state gives precisely the even positions.

- (8 pts) (b) Prove that *no* DFA correctly determines, for an arbitrary command sequence s , whether Pac-Man will end up back at home (at the origin) after completing the commands.

Solution: Let

$$H = \{s \in \{L, R, U, D\}^* : \#_L(s) = \#_R(s) \text{ and } \#_U(s) = \#_D(s)\}.$$

Suppose, for contradiction, that a DFA M with s states decides H . Consider the $s + 1$ strings

$$\varepsilon, R, R^2, \dots, R^s.$$

After reading these strings, M has visited only s possible states, so two of them, say R^i and R^j with $0 \leq i < j \leq s$, leave M in the same state. Append the same suffix L^i to both strings. Since the two computations start their suffix in the same state, M must give the same answer on $R^i L^i$ and $R^j L^i$.

However, $R^i L^i$ returns to the origin and belongs to H , while $R^j L^i$ ends at horizontal coordinate $j - i > 0$ and does not belong to H . This contradiction shows that no finite-state DFA can decide the homecoming language.

- (4 pts) (c) Give an algorithm using pseudocode that takes as input an arbitrary command sequence s and determines whether Pac-Man will end up back at home after completing the sequence. Justify its correctness, but you do not need to analyze the running time.

Solution:

```

1:  $(x, y) \leftarrow (0, 0)$ 
2: for all  $c$  in  $s$  from left to right do
3:   if  $c = L$  then  $x \leftarrow x - 1$ 
4:   else if  $c = R$  then  $x \leftarrow x + 1$ 
5:   else if  $c = U$  then  $y \leftarrow y + 1$ 
6:   else  $y \leftarrow y - 1$ 
7: if  $x = 0$  and  $y = 0$  then return accept
8: elsereturn reject

```

Algorithm 3: AtHome(s)

After reading any prefix of s , an induction on the prefix length shows that the variables (x, y) equal Pac-Man's actual coordinates: the claim is true initially, and each command changes exactly the corresponding coordinate by one in the pseudocode and on the grid. Consequently, at the end the algorithm accepts exactly when the final position is $(0, 0)$, which proves correctness.

8. Decision problems and languages.

Recall that the goal of a *decision problem* is to determine whether a given input “object” has a certain “property”, e.g., determine whether a given integer is prime, or whether a given string is a palindrome. In class, we said that any decision problem is equivalent to a membership problem for a corresponding *language*. That is, determining whether a given object has the property is equivalent to determining whether its encoding (as a string) is a member of the language.

For each of the following decision problems,

- (i) Define a reasonable (finite) alphabet Σ
- (ii) Give an encoding (over the alphabet) of a representative input object
- (iii) Analyze the length of the encoding
- (iv) Define a language L for the corresponding decision problem

Use the notation $\langle X \rangle$ for the encoding of object X as a string over the alphabet.

- (0 pts) (a) “Does a given non-negative integer k have 3 as its last digit, when written in base 10?”

Solution:

- (i) A reasonable alphabet consists of the decimal digits, $\Sigma = \{0, \dots, 9\}$.
- (ii) Encode k by its usual base-10 representation. For example, $\langle 47 \rangle = 47$.
- (iii) The length is the number of digits, $\Theta(\log(k + 2))$.
- (iv) $L = \{\langle k \rangle : k \bmod 10 = 3\}$.

- (0 pts) (b) “Is a given array of non-negative integers sorted?”

Solution:

- (i) Use $\Sigma = \{0, \dots, 9, [,], ,\}$.
- (ii) Encode $A = [a_1, \dots, a_n]$ as $\langle A \rangle = [\langle a_1 \rangle, \dots, \langle a_n \rangle]$; for example, $\langle [1, 2, 3, 4] \rangle = [1, 2, 3, 4]$.
- (iii) The length is $\Theta(n + \sum_{i=1}^n \log(a_i + 2))$, including the brackets and separators.
- (iv) $L_{\text{sorted}} = \{\langle A \rangle : a_1 \leq a_2 \leq \dots \leq a_n\}$.

- (4 pts) (c) “Given an array S of integers, are all its elements distinct?”

Solution:

- (i) Use the finite alphabet $\Sigma = \{0, \dots, 9, -, [,], ,\}$, with canonical signed decimal encodings (no leading zeroes except for zero itself).
- (ii) For $S = [s_1, \dots, s_n]$, encode $\langle S \rangle = [\langle s_1 \rangle, \dots, \langle s_n \rangle]$. For example, $\langle [-3, 0, 12] \rangle = [-3, 0, 12]$.
- (iii) The length is $\Theta(n + \sum_{i=1}^n \log(|s_i| + 2))$.
- (iv) The language is

$$L_{\text{distinct}} = \{\langle S \rangle : \forall i \neq j, s_i \neq s_j\}.$$

- (4 pts) (d) “Is a given undirected graph complete?”

Solution:

(i) Let $\Sigma = \{0, 1, \#\}$.

(ii) For a graph on vertex set $\{1, \dots, n\}$, write

$$\langle G \rangle = 1^n \# b_{11} b_{12} \cdots b_{1n} b_{21} \cdots b_{nn},$$

where $b_{ij} = 1$ exactly when $\{i, j\}$ is an edge. The $n \times n$ adjacency matrix is listed row by row.

(iii) The encoding length is $n + 1 + n^2 = \Theta(n^2)$.

(iv) Define

$$L_{\text{complete}} = \{\langle G \rangle : \langle G \rangle \text{ is well formed and } b_{ii} = 0 \text{ and } b_{ij} = 1 \text{ for every } i \neq j\}.$$

This is exactly the language of complete simple undirected graphs.

(4 pts)

(e) “Does a given C++ program terminate when run on a given input?”

Solution:

(i) Let Σ be the finite set of ASCII characters together with a delimiter $\#$.

(ii) Encode a pair (P, x) by

$$\langle P, x \rangle = \text{bin}(|P|) \# Px,$$

where P is the ASCII source of the program and x is its input. The length prefix tells the decoder exactly how many characters belong to P , so the delimiter need not be absent from the source or input.

(iii) The length is $\Theta(|P| + |x| + \log(|P| + 1))$, which is $\Theta(|P| + |x|)$ when the program length is at least logarithmic in its binary length.

(iv) The corresponding language is

$$L_{\text{HALT}} = \{\langle P, x \rangle : \text{the C++ program } P \text{ eventually halts when run on input } x\}.$$

Ill-formed encodings are simply excluded from the language.

9. Trophy testing (optional extra credit).

(2 EC pts)

(a) Suppose you are given two blocks of Waterford Crystal and allowed no more than d drops, from heights of your choice (which can depend on the results of previous drops). What is the largest value of $n = n(d)$ for which you can determine the LBH among the $n + 1$ different possibilities? Describe your method clearly and concisely in English, give a recurrence and base case for $n(d)$, and give an exact (not asymptotic) closed-form solution.

Solution: Let $n(d, b)$ be the largest number of tested heights that can be distinguished with d drops and b blocks. With one block, $n(d, 1) = d$, since the heights must be tested from low to high. Also $n(0, 2) = 0$. If the first block is dropped at some height, then a break leaves at most $n(d - 1, 1)$ lower heights to distinguish, while a survival leaves at most $n(d - 1, 2)$ higher heights. The tested height itself is a third possibility, so

$$n(d, 2) = n(d - 1, 1) + 1 + n(d - 1, 2) = (d - 1) + 1 + n(d - 1, 2) = d + n(d - 1, 2).$$

The matching strategy drops the first block at successive gaps $d, d - 1, d - 2, \dots, 1$. In other words, the drop heights are

$$d, \quad d + (d - 1), \quad d + (d - 1) + (d - 2), \quad \dots,$$

and, if the first block breaks, the second block linearly tests the remaining heights in the preceding gap. If the first block survives all drops, the LBH is greater than the final tested height. Solving the recurrence gives

$$n(d) = n(d, 2) = \sum_{j=1}^d j = \frac{d(d+1)}{2}.$$

(1 EC pt)

- (b) Give a short explanation why your algorithm is *optimal*; i.e., why *any algorithm* that uses 2 blocks and at most d drops cannot be guaranteed to work for a value larger than $n = n(d)$, for the $n(d)$ you gave in the previous part.

Solution: Consider the first drop of any strategy. If the block breaks, the strategy has one block and at most $d - 1$ drops left, so it can distinguish at most $n(d - 1, 1) = d - 1$ lower heights, in addition to the first-drop height itself. If the block survives, it has two blocks and at most $d - 1$ drops left, so it can distinguish at most $n(d - 1, 2)$ higher heights. Therefore every strategy satisfies

$$n \leq n(d - 1, 1) + 1 + n(d - 1, 2) = n(d, 2).$$

Induction on d makes this an upper bound for every adaptive strategy, while the decreasing-gap strategy in the previous part attains the bound. Hence it is optimal.